Introspection of Deep Learning Models Trained on Brain fMRI Data

Pavel Popov
ppopov1@student.gsu.edu

Ufuk Seçilmiş
Georgia State University
usecilmis1@student.gsu.edu

Selim Süleymanoğlu
ssleymanoglu1@student.gsu.edu

Abstract

Although deep learning (DL) models are capable of good classification on brain fMRI data, their complexity often hinders data explainability. As a result, finding tools to interpret fMRI data is crucial for advancing our understanding of brain functions. In this work, we investigate the ability of a few DL models to reveal anomalies in synthetic data, with the goal of utilizing these models for brain fMRI data explainability.

1. Introduction

In recent years deep learning methods have been extensively used in brain image analysis. Deep learning (DL) models, while being inspired by the organic neural mechanics, in return proved to be highly effective in establishing causal connections between brain regions and showed decent results on the classification of brain disorders from brain imaging data. However, despite this classification success, the complexity of DL models makes them effectively 'black boxes' when it comes to the problem of pinpointing the anomalies in the data which led to a particular classification decision. Solving this problem would be useful for understanding brain processes and disorders. For this project, we want to analyze the ability of a few general and brain-focused DL models, trained for classification on some synthetic and brain imaging datasets, to produce sound saliency maps.

The code for this project can be found at `https://github.com/paavalipopov/intro-dl-project`.

2. Datasets

We use two open-access brain fMRI datasets. ABIDE (release 1) [2] dataset consists of 569 samples, with 255 samples of class 0 (patients with autism spectrum disorders) and 314 samples of class 1 (healthy controls). COBRE [1] dataset consists of 157 samples, with 68 samples of class 0 (patients with schizophrenia) and 89 samples of class 1 (healthy controls). Both datasets are processed to remove noise and transform 3D brain images into 1D vectors of ICA components. This way, for each sample from these datasets we had a time series of signals extracted from ICA regions.

However, real-world data alone is hardly usable for assessment of the model's ability to detect anomalies, since the results are difficult to verify. For this reason, we crafted two synthetic datasets with manually introduced anomalies.

Synthetic1 dataset consists of 200 samples, 100 of class 0 and 100 of class 1. Each sample consists of 53 time series of the form

$$f_{i,t} = -a_i \times |T - t| + b_i + \sin(\omega_i t + \phi_i) + \mathcal{N}(0,1), \quad i \in \{1, 2, \dots 53\},\ t \in \{0, 1, \dots 139\}, \tag{1}$$

where $a_i \in [1/14, 10/14]$, $b_i \in [-4, 4]$, $\omega_i \in [\frac{2\pi}{140}, \frac{140\pi}{140}]$, and $\phi_i \in [0,\ 2\pi]$ are chosen randomly for each i. T depends on the class: for class 0 T is always 70; for class 1 $T \in [50, 60] \cup [80, 90]$.

Synthetic2 dataset also consists of 200 samples, 100 of class 0 and 100 of class 1. Each sample of class 0 consists of 53 time series of the form

$$f_{i,t} = \begin{cases} a_i \times (t - T) + b_i + \sin(\omega_i t + \phi_i) + \mathcal{N}(0,1), \\ t \bmod 3 = 0; \\ a_i \times (T - t) + b_i + \sin(\omega_i t + \phi_i) + \mathcal{N}(0,1), \\ \text{else}, \end{cases} \quad i \in \{1, 2, \dots 53\},\ t \in \{0, 1, \dots 139\}, \tag{2}$$

where $a_i \in [1/14, 10/14]$, $b_i \in [-4, 4]$, $\omega_i \in [\frac{2\pi}{140}, \frac{140\pi}{140}]$, $\phi_i \in [0,\ 2\pi]$ are chosen randomly for each i, $T = 70$. T depends on the class: for class 0 T is always 70; for class 1 $T \in [50, 60] \cup [80, 90]$. Each sample of class 1 has the same form; however, if $t = 2i$, the sign of $f_{i,t}$ is flipped.

3. Methods

3.1. Models

We implemented a few models working with time-series data: more general LSTM and Transformer models (and

their variations), and a DICE model, designed for the brain fMRI data.

3.1.1 LSTM and Mean LSTM

We implemented LSTM and Mean LSTM models for classification, using the PyTorch LSTM implementation with tunable embedding size and dropout hyperparameters. Bidirectionality was also treated as a tunable hyperparameter. Both models consist of an LSTM block, which extracts the embeddings of the input data, and an FC layer, which uses these embeddings for class predictions. The difference between models is how they treat LSTM embeddings. In 'vanilla' LSTM design, the last (for unidirectional LSTM) or the concatenated first and last (for bidirectional LSTM) embeddings are passed to the FC layer. In Mean LSTM, the mean of LSTM embeddings is used instead. Using the mean embedding showed better classification results on the real-world data.

3.1.2 Transformer and Mean Transformer

In Transformer and Mean Transformer models we utilized the PyTorch implementation of multi-layer transformer encoder from [4] with tunable number of layers, number of heads, and the embedding size. Transformer encoder is preceded by an FC layer, followed by ReLU, that transforms the input data feature size at each time point to the embedding size of the encoder. Encoder output embeddings are passed to an FC layer to produce class predictions. Similarly to LSTM models, we use the mean of encoder output embeddings for classification in the Mean Transformer model, and the first embedding in the Transformer model. Using the mean embedding instead of the single first embedding showed better classification results on the real-world data.

3.1.3 DICE

DICE model [3] is designed specifically for the brain fMRI classification with interpretability in mind.

In the DICE model, each feature‘s time series is passed through the same bidirectional LSTM to infer its embedding at each time point. These embeddings are then passed through the self-attention mechanism [4] to retrieve the spatial relations between features at each time point. Finally, the model utilizes a global temporal attention module to derive the global connectivity graph, which can be used for interpretability, or a downstream classification task.

3.2. Training

We used stratified 5-fold Cross Validation (CV) to tune and test each model on each dataset. For each test fold, we first fine-tuned the model in a nested 5-fold CV on the training data using 50 rounds of Random Search. Then, we trained the fine-tuned model on 10 different train/validation splits of the training data. Finally, we tested the model with the smallest loss on the validation data on the test fold. In the end, for each model-dataset pair we had 50 test results.

We used AUROC metric as a measure of the test performance. We ensured that for each dataset the train/validation/test splits remained the same across the experiments with different models.

3.3. Introspection

We utilized Captum package for inspection of the trained models. For each model-dataset pair we picked a model trained for the first test fold that showed the highest test AUROC score and plotted the saliency maps on all of the test samples. For saliency maps, we used Saliency, Integrated Gradients, and Integrated Gradients + Noise Tunnel methods from the Captum package.

4. Results

4.1. Classification

Figure 1 shows the classification results of the trained models on different datasets. As can be seen, Mean LSTM and Mean Transformer perform better than their vanilla counterparts on the real-world data. Nonetheless, all models show rather modest results on the real world-data. This can be, to some extent, explained by the inconsistency in the data, since samples in the ABIDE and COBRE datasets were collected on different sites. On the synthetic datasets we see that transformer-based models managed to detect anomalies in both cases. Interestingly, both vanilla LSTM and Mean LSTM failed at classifying Synthetic1 dataset, yet only Mean LSTM failed at Synthetic2 dataset. The DICE model managed to detect anomalies only on the Synthetic2 dataset, however with a peculiar variance.

4.2. Saliency maps

Figure 2 shows the saliency maps based on test samples of each class from different datasets. As can be seen from the maps on Synthetic1 dataset, Transformer and Mean Transformer successfully detected the anomalies, while other models seem to focus on unrelated data. From Synthetic2 maps we can see that models that did well on this dataset have focused on anomalies in only a few feature time-series; however, it was enough for classification.

From the saliency maps based on real world data we can see that vanilla LSTM and Transformer models focused primarily on the time point which corresponds to the output embedding used for classification. Yet from the Fig. 1 we see that is was enough for classification on ABIDE (but not COBRE), however with AUROC smaller than the AUROC of Mean models. Mean LSTM and Transformer mod-

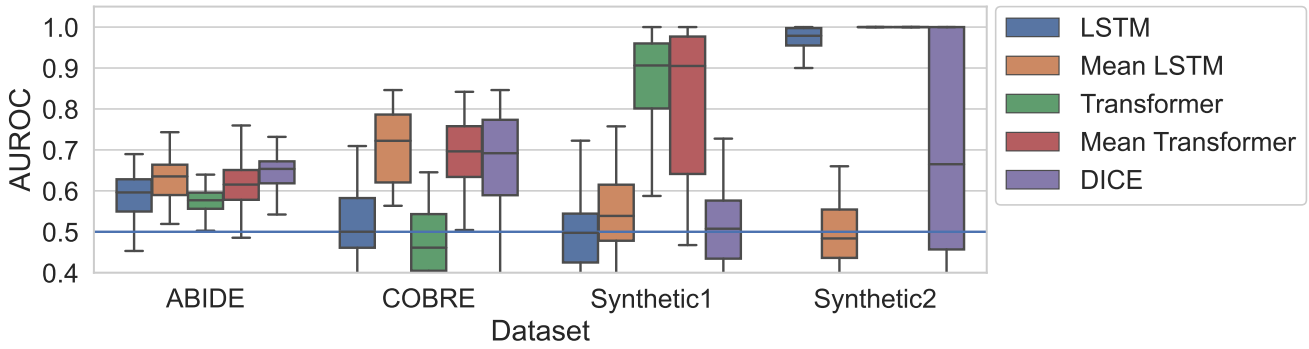

Figure 1. Classification results of the studied models. The blue line at AUC $= 0.5$ indicates a random choice baseline. Mean LSTM and Transformer tend to classify the real world data better than their vanilla counterparts. On the synthetic data it seems as if the ability to detect anomalies depends on the model architecture.

els seem to focus on the same features on each time point on ABIDE dataset. However, we see that the same models have some variation of attention over time. The DICE model seem to exhibit the same attention patterns as Mean models, however less explicitly.

5. Conclusions

We came up with several findings and conclusions:

1. We need more synthetic datasets for the sake of completeness. From the saliency maps based on the real world data we see that we need to craft a dataset with anomalies, introduced within a few features‘ time series, not on just a few time points.

2. Additionally, our research lacks a measure of similarity between saliency maps of different models. Finding one would help to better understand the relationship between the models and the significance of found anomalies.

3. Finally, while neither our models classified the brain fMRI data perfectly, nor they provided easily interpretable saliency maps, we found an intriguing clue regarding the difference between anomalies in the ABIDE and COBRE datasets. Judging by the saliency maps, we can assume that classifying the Autism spectrum disorder (ABIDE dataset) does not require data on brain activity over time, but a single snapshot would be enough. At the same time, it does not seem to apply to Schizophrenia (COBRE dataset), as we see the attention difference over time in the saliency maps, which also correlates with the classification success of different models. This assumption might have ground in the nature of these disorders and is worthy of further research.

6. Peer Evaluation

Our division of labour was as follows:

- Pavel implemented the models and the training scripts, and also put together Ufuk's and Selim's code.
- Ufuk investigated the approaches for model introspection, and implemented the code for saliency maps.
- Selim investigated approaches and conventions for synthetic data generation, and constructed the two synthetic datasets in Python according to formulae provided in Section 1.

Ufuk	Selim	Myself
20	20	20

Table 1. Self-Peer Evaluation Table

References

[1] Mustafa S Çetin et al. Thalamus and posterior temporal lobe show greater inter-network connectivity at rest and across sensory paradigms in schizophrenia. *Neuroimage*, 97:117–126, Apr. 2014.

[2] A Di Martino et al. The autism brain imaging data exchange: towards a large-scale evaluation of the intrinsic brain architecture in autism. *Mol Psychiatry*, 19(6):659–667, June 2013.

[3] Usman Mahmood et al. Through the looking glass: Deep interpretable dynamic directed connectivity in resting fmri. *NeuroImage*, 264:119737, 2022.

[4] Ashish Vaswani et al. Attention is all you need, 2017.

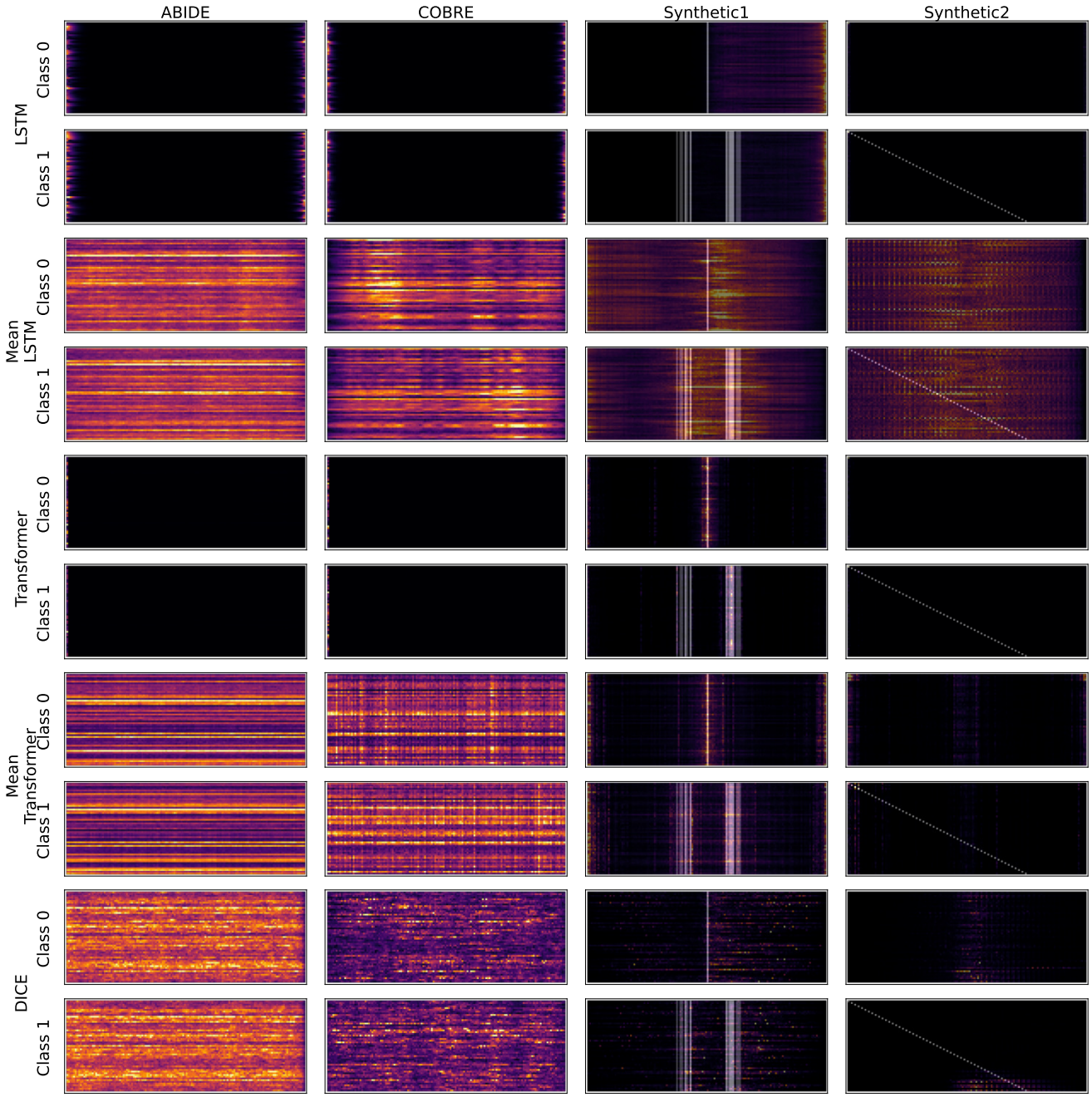

Figure 2. Saliency maps, plotted for the trained models on test samples of each class from different datasets. Captum's 'Saliency' method was used. The thin light-colored lines on the synthetic datasets indicate the locations of the manually introduced anomalies.

A. Appendix

A.1. Code and Datasets

The source code for the projects and scripts for donwloading/generating datasets can be found at `https://github.com/paavalipopov/intro-dl-project`.